LAB ASSIGNMENT – 12.5

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description #1 (Sorting – Merge Sort Implementation)

• Task: Use AI to generate a Python program that implements

the Merge Sort algorithm.

• Instructions:

o Prompt AI to create a function merge_sort(arr) that

sorts a list in ascending order.

o Ask AI to include time complexity and space

complexity in the function docstring.

o Verify the generated code with test cases.

• Expected Output:

o A functional Python script implementing Merge Sort

with proper documentation.

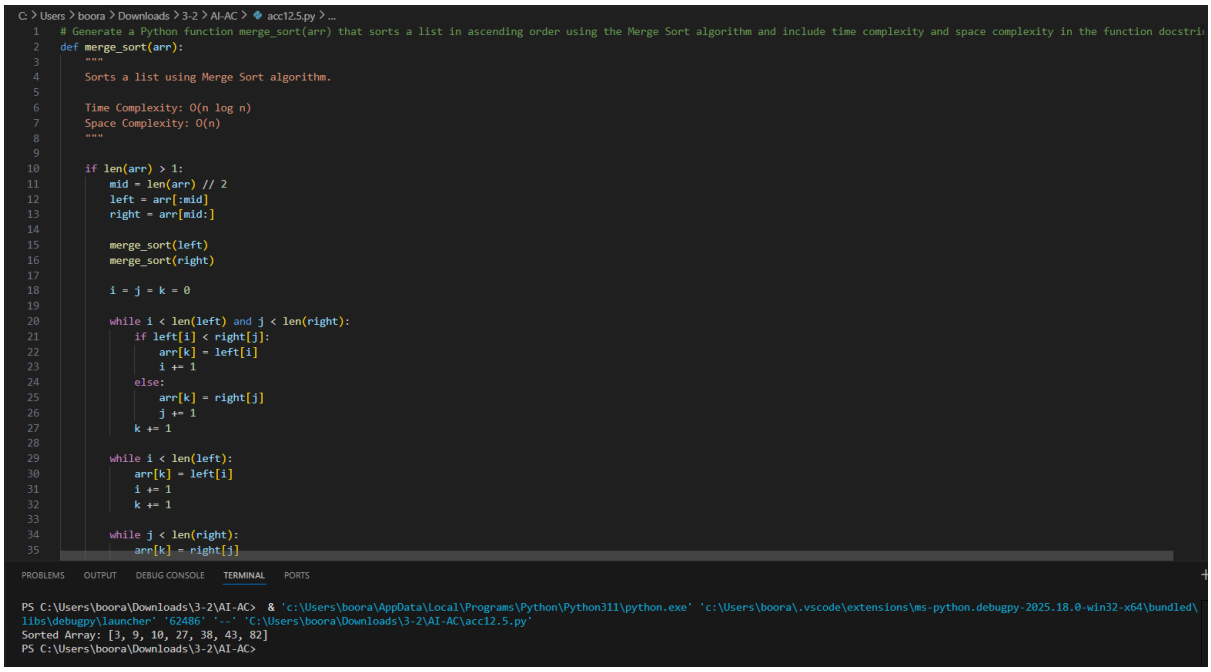
Code:

```
# Generate a Python function merge_sort(arr) that sorts a list in ascending order using the Merge Sort algorithm and include time complexity and space complexity in the function docstring

def merge_sort(arr):
    """
    Sorts a list using Merge Sort algorithm.

    Time Complexity: O(n log n)
    Space Complexity: O(n)
    """
```

```
if len(arr) > 1:
    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    merge_sort(left)
    merge_sort(right)

    i = j = k = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        arr[k] = right[j]
        j += 1
```

```
        k += 1

    return arr

# Example usage
arr = [38, 27, 43, 3, 9, 82, 10]
print("Sorted Array:", merge_sort(arr))
```

Output:



Justification:

Merge Sort is an efficient divide-and-conquer sorting algorithm. It splits the list into smaller sublists and merges them in sorted order. It guarantees *O(n log n)* time complexity and works well for large datasets.

Task Description #2 (Searching – Binary Search with AI

Optimization)

• Task: Use AI to create a binary search function that finds a

target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr,

target) returning the index of the target or -1 if not

found.

o Include docstrings explaining best, average, and

worst-case complexities.

o Test with various inputs.

• Expected Output:

o Python code implementing binary search with AI-

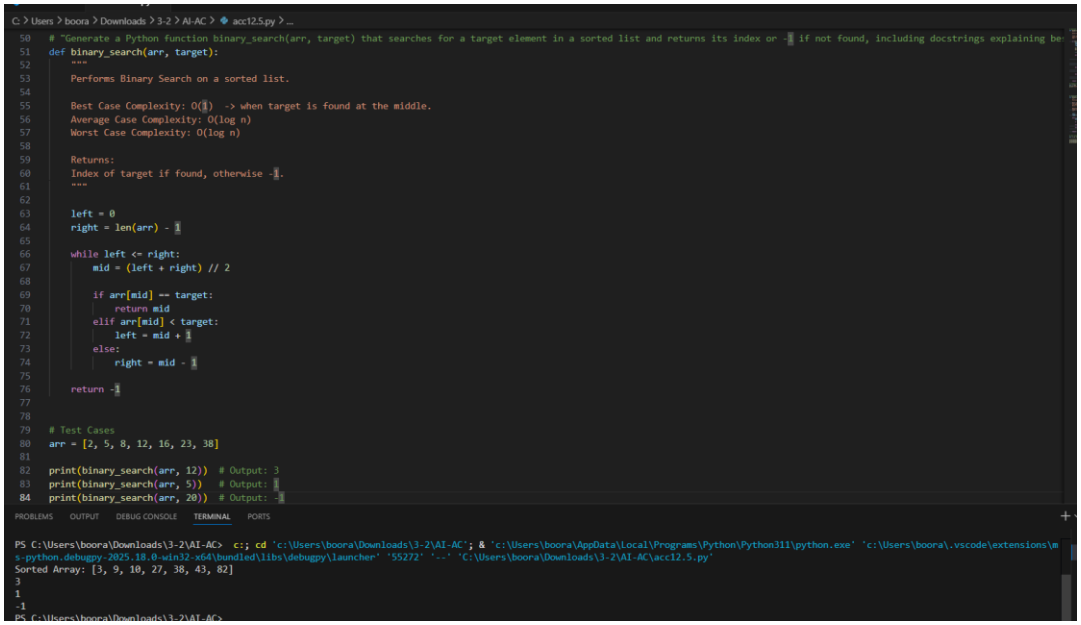
generated comments and docstrings.

Code:

```
# "Generate a Python function binary_search(arr, target) that searches for a target element in a sorted list and returns its index or -1 if not found, including docstrings explaining best, average, and worst-case time complexities
def binary_search(arr, target):
    """
    Performs Binary Search on a sorted list.

    Best Case Complexity: O(1)  -> when target is found at the middle.
    Average Case Complexity: O(log n)
    Worst Case Complexity: O(log n)

    Returns:
    Index of target if found, otherwise -1.
```

```
    """

    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1


# Test Cases
arr = [2, 5, 8, 12, 16, 23, 38]

print(binary_search(arr, 12))  # Output: 3
print(binary_search(arr, 5))   # Output: 1
print(binary_search(arr, 20))  # Output: -1
```

Output:



Justification:

Binary Search is efficient for searching elements in a *sorted list. It repeatedly divides the list into halves until the target element is found. This reduces the search time to **O(log n)*, making it faster than linear search.

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

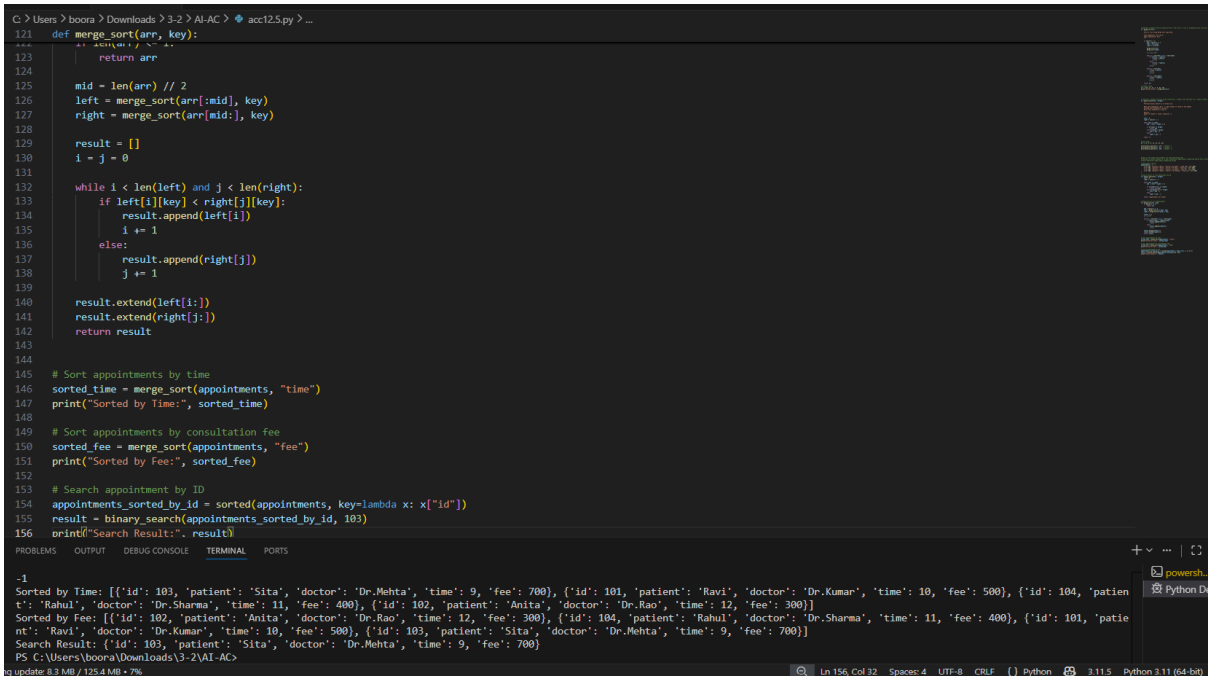
Code:

```
# Here is the **one-line prompt** for Task Description #3:

#"Generate a Python program for a Smart Healthcare Appointment Scheduling System that stores appointment ID, patient name, doctor name, appointment time, and consultation fee, uses Binary Search to find appointments by ID, and uses Merge Sort to sort appointments by time or consultation fee with proper comments and explanations."**

# Smart Healthcare Appointment Scheduling System


# Appointment records

appointments = [

  {"id":101, "patient":"Ravi", "doctor":"Dr.Kumar", "time":10, "fee":500},

  {"id":102, "patient":"Anita", "doctor":"Dr.Rao", "time":12, "fee":300},

  {"id":103, "patient":"Sita", "doctor":"Dr.Mehta", "time":9, "fee":700},

  {"id":104, "patient":"Rahul", "doctor":"Dr.Sharma", "time":11, "fee":400}

]


# Binary Search to find appointment by ID

def binary_search(arr, target):

  left = 0

  right = len(arr) - 1


  while left <= right:

    mid = (left + right) // 2


    if arr[mid]["id"] == target:

      return arr[mid]

    elif arr[mid]["id"] < target:
```

```
            left = mid + 1
        else:
            right = mid - 1

    return "Appointment not found"


# Merge Sort to sort appointments
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
```

```
    result.extend(right[j:])
    return result


# Sort appointments by time
sorted_time = merge_sort(appointments, "time")
print("Sorted by Time:", sorted_time)

# Sort appointments by consultation fee
sorted_fee = merge_sort(appointments, "fee")
print("Sorted by Fee:", sorted_fee)

# Search appointment by ID
appointments_sorted_by_id = sorted(appointments, key=lambda x: x["id"])
result = binary_search(appointments_sorted_by_id, 103)
print("Search Result:", result)
```

Output:



Justification:

Binary Search is used to quickly find appointments using *appointment ID* in sorted records. Merge Sort efficiently organizes appointments based on *time or consultation fee*. These algorithms improve system performance when handling large appointment datasets.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket

ID, passenger name, train number, seat number, and travel date. The

system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python.

Code:

"Generate a Python program for a Railway Ticket Reservation System that stores ticket ID, passenger name, train number, seat number, and travel date, uses Binary Search to find tickets by ticket ID, and uses Merge Sort to sort bookings by travel date or seat number with proper comments and documentation."

Railway Ticket Records

```
tickets = [
  {"id":201, "passenger":"Ramesh", "train":1201, "seat":15, "date":"2026-04-01"},
  {"id":202, "passenger":"Sita", "train":1205, "seat":10, "date":"2026-03-28"},
```

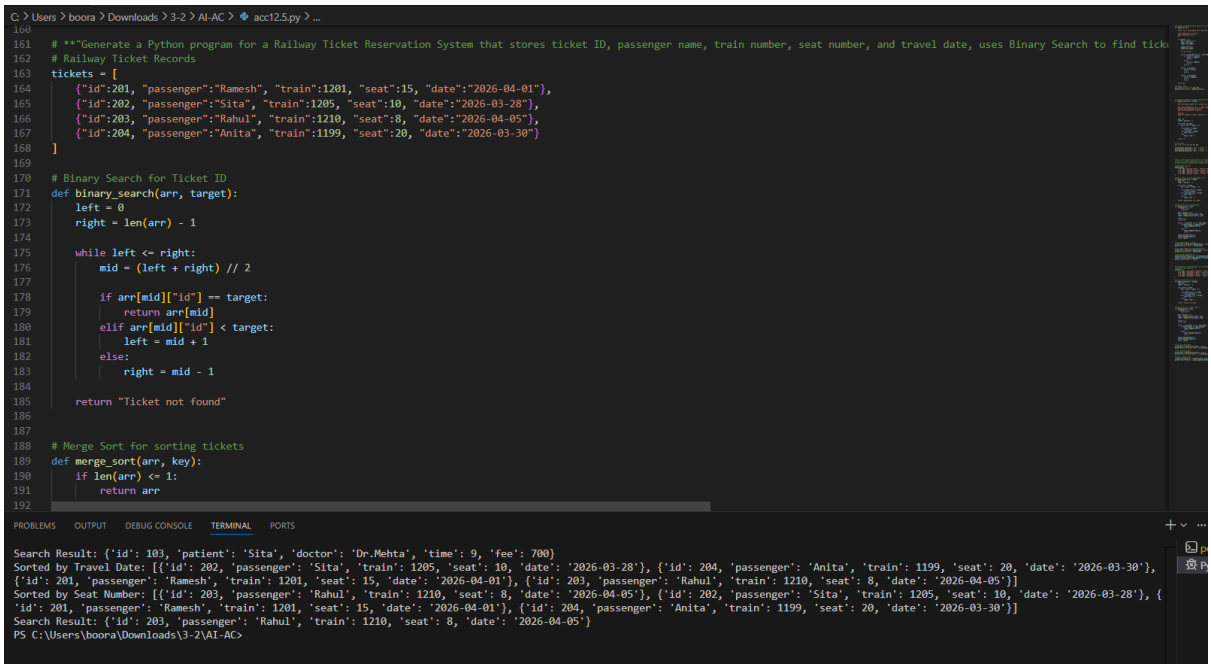
```
  {"id":203, "passenger":"Rahul", "train":1210, "seat":8, "date":"2026-04-05"},
  {"id":204, "passenger":"Anita", "train":1199, "seat":20, "date":"2026-03-30"}
]

# Binary Search for Ticket ID
def binary_search(arr, target):
  left = 0
  right = len(arr) - 1

  while left <= right:
    mid = (left + right) // 2

    if arr[mid]["id"] == target:
      return arr[mid]
    elif arr[mid]["id"] < target:
      left = mid + 1
    else:
      right = mid - 1

  return "Ticket not found"



# Merge Sort for sorting tickets
def merge_sort(arr, key):
  if len(arr) <= 1:
    return arr

  mid = len(arr) // 2
```

```
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# Sort by travel date
sorted_date = merge_sort(tickets, "date")
print("Sorted by Travel Date:", sorted_date)

# Sort by seat number
sorted_seat = merge_sort(tickets, "seat")
print("Sorted by Seat Number:", sorted_seat)

# Search ticket by ID
```

tickets_sorted_by_id = sorted(tickets, key=lambda x: x["id"])

print("Search Result:", binary_search(tickets_sorted_by_id, 203))

Output:



Justification:

Binary Search helps quickly locate ticket details using *ticket ID. Merge Sort is used to arrange booking records by **travel date or seat number*. These algorithms ensure efficient searching and sorting in railway reservation systems.

Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details

including student ID, room number, floor, and allocation date. The

system needs to:

1. Search allocation details using student ID.

2. Sort records based on room number or allocation date.

Student Task

• Use AI to suggest optimized algorithms.

• Justify the selections.

• Implement the solution in Python.

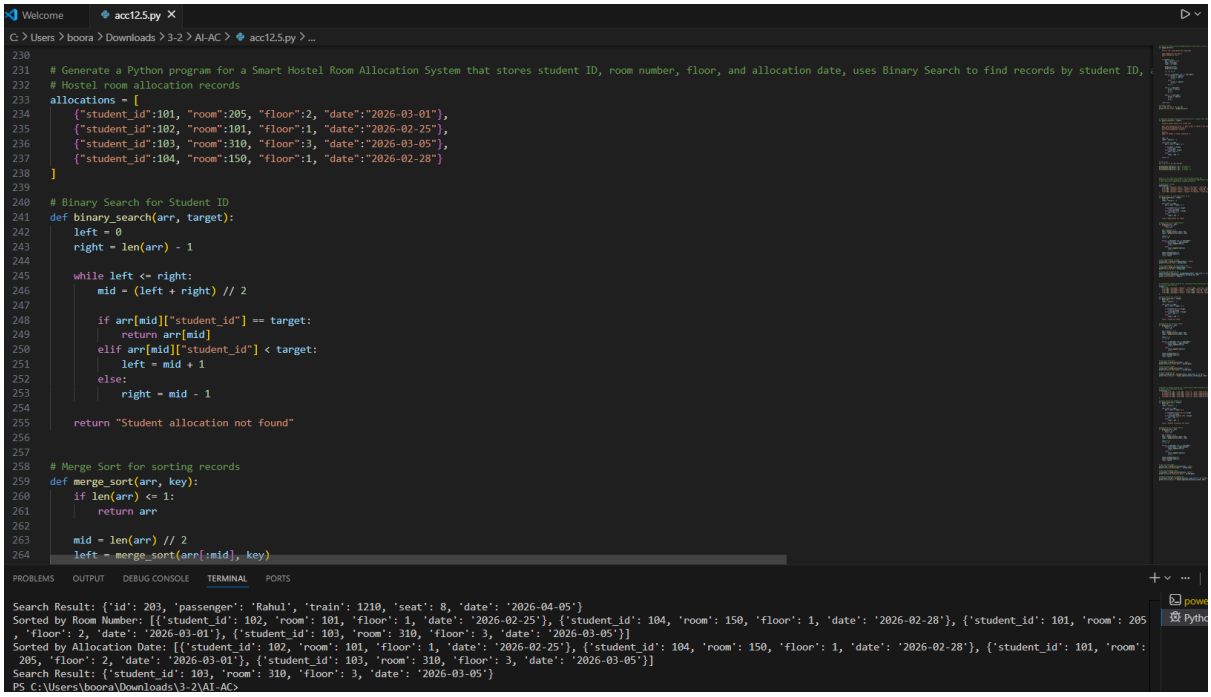
Code:

```
# Generate a Python program for a Smart Hostel Room Allocation System that stores student ID, room number, floor, and allocation date, uses Binary Search to find records by student ID, and uses Merge Sort to sort records by room number or allocation date with proper comments

# Hostel room allocation records
allocations = [
  {"student_id":101, "room":205, "floor":2, "date":"2026-03-01"},
  {"student_id":102, "room":101, "floor":1, "date":"2026-02-25"},
  {"student_id":103, "room":310, "floor":3, "date":"2026-03-05"},
  {"student_id":104, "room":150, "floor":1, "date":"2026-02-28"}
]

# Binary Search for Student ID
def binary_search(arr, target):
  left = 0
  right = len(arr) - 1

  while left <= right:
    mid = (left + right) // 2

    if arr[mid]["student_id"] == target:
      return arr[mid]
    elif arr[mid]["student_id"] < target:
```

```
            left = mid + 1
        else:
            right = mid - 1

    return "Student allocation not found"



# Merge Sort for sorting records
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
```

```
    result.extend(right[j:])
    return result


# Sort by room number
sorted_room = merge_sort(allocations, "room")
print("Sorted by Room Number:", sorted_room)

# Sort by allocation date
sorted_date = merge_sort(allocations, "date")
print("Sorted by Allocation Date:", sorted_date)

# Search allocation by student ID
allocations_sorted = sorted(allocations, key=lambda x: x["student_id"])
print("Search Result:", binary_search(allocations_sorted, 103))
```

Output:



Justification:

Binary Search efficiently finds student allocation details using *student ID. Merge Sort organizes records by **room number or allocation date*. This improves data management and retrieval in hostel systems.

Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title,

genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.

2. Sort movies based on rating or release year.

Student Task

• Recommend searching and sorting algorithms using AI.

• Justify the chosen algorithms.

• Implement Python functions.

Code:

Generate a Python program for an Online Movie Streaming Platform that stores movie ID, title, genre, rating, and release year, uses Binary Search to find movies by movie ID, and uses Merge Sort to sort movies by rating or release year with proper comments

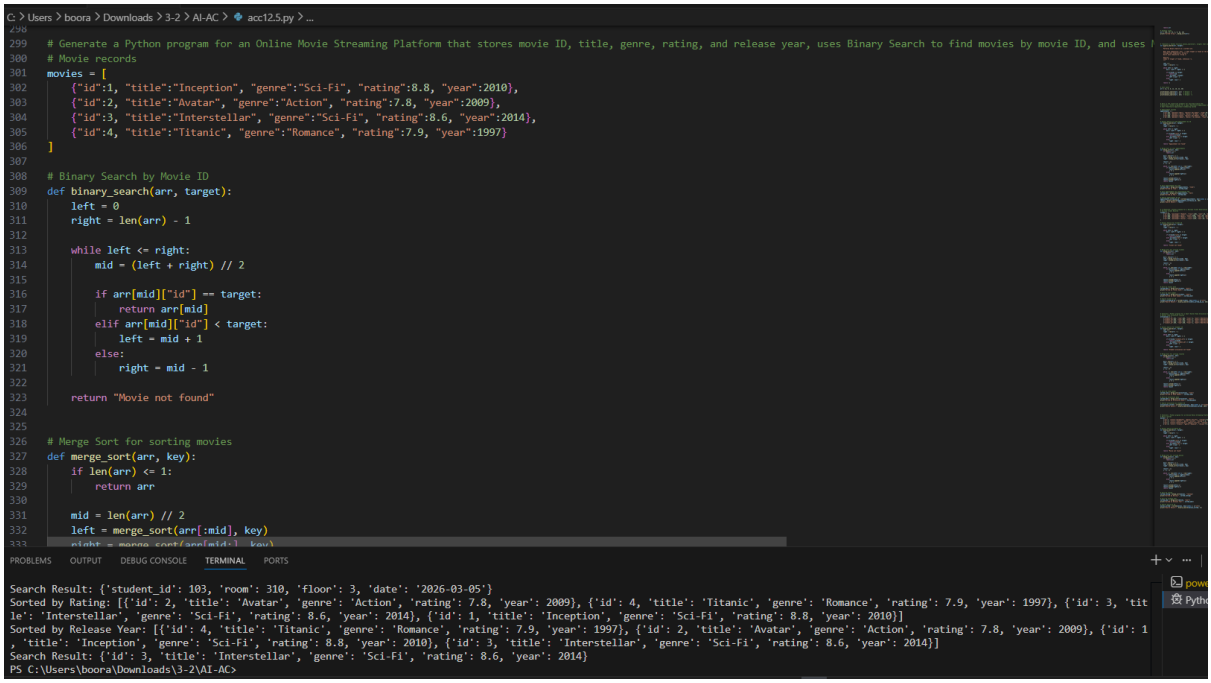
Movie records

```
movies = [
  {"id":1, "title":"Inception", "genre":"Sci-Fi", "rating":8.8, "year":2010},
  {"id":2, "title":"Avatar", "genre":"Action", "rating":7.8, "year":2009},
  {"id":3, "title":"Interstellar", "genre":"Sci-Fi", "rating":8.6, "year":2014},
  {"id":4, "title":"Titanic", "genre":"Romance", "rating":7.9, "year":1997}
]
```

```
# Binary Search by Movie ID
def binary_search(arr, target):
    left = 0
    right = len(arr) - 1

    while left <= right:
        mid = (left + right) // 2

        if arr[mid]["id"] == target:
            return arr[mid]
        elif arr[mid]["id"] < target:
            left = mid + 1
        else:
            right = mid - 1

    return "Movie not found"



# Merge Sort for sorting movies
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)
```

```
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# Sort by rating
sorted_rating = merge_sort(movies, "rating")
print("Sorted by Rating:", sorted_rating)

# Sort by release year
sorted_year = merge_sort(movies, "year")
print("Sorted by Release Year:", sorted_year)

# Search movie by ID
movies_sorted = sorted(movies, key=lambda x: x["id"])
print("Search Result:", binary_search(movies_sorted, 3))
```

Output:



Justification:

Binary Search helps quickly search movie records using *movie ID. Merge Sort is suitable for arranging movies by **rating or release year*. These algorithms allow efficient handling of large movie databases.

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop

name, soil moisture level, temperature, and yield estimate. Farmers

need to:

1. Search crop details using crop ID.

2. Sort crops based on moisture level or yield estimate.

Student Task

• Use AI-assisted reasoning to select algorithms.

• Justify algorithm suitability.

• Implement searching and sorting in Python.

Code:

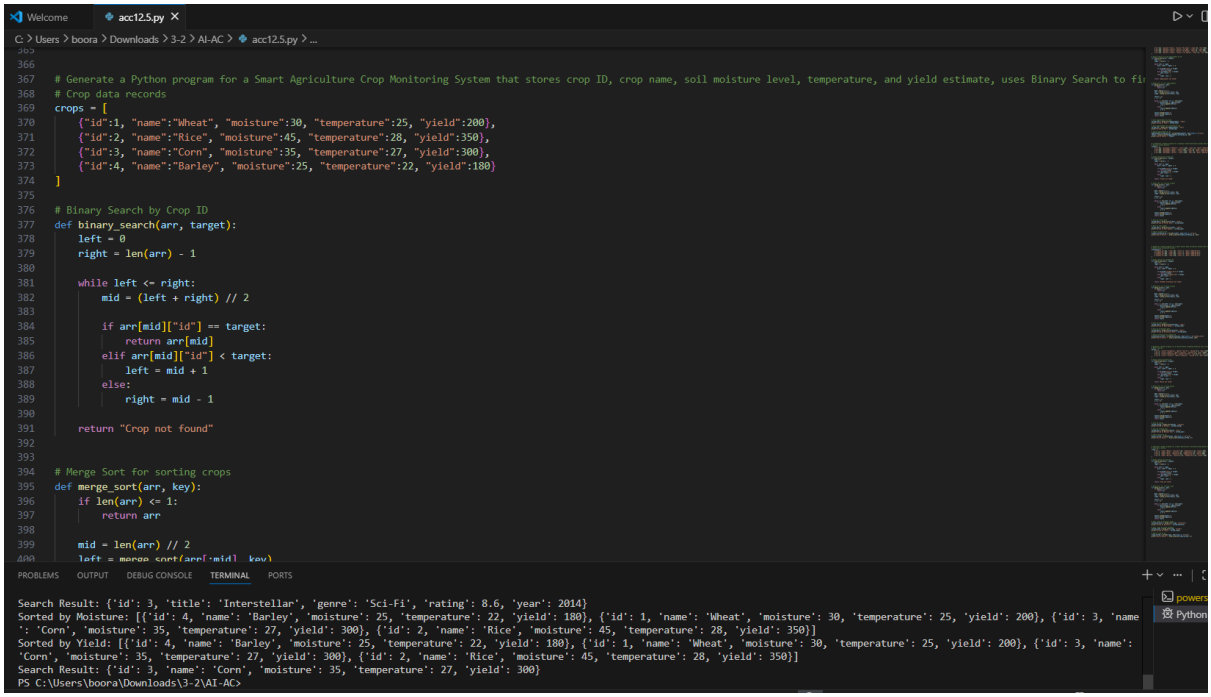
Generate a Python program for a Smart Agriculture Crop Monitoring System that stores crop ID, crop name, soil moisture level, temperature, and yield estimate, uses Binary Search to find crops by crop ID, and uses Merge Sort to sort crops by moisture level or yield estimate with proper comments

```
# Crop data records
crops = [
  {"id":1, "name":"Wheat", "moisture":30, "temperature":25, "yield":200},
  {"id":2, "name":"Rice", "moisture":45, "temperature":28, "yield":350},
  {"id":3, "name":"Corn", "moisture":35, "temperature":27, "yield":300},
  {"id":4, "name":"Barley", "moisture":25, "temperature":22, "yield":180}
]

# Binary Search by Crop ID
def binary_search(arr, target):
  left = 0
  right = len(arr) - 1

  while left <= right:
    mid = (left + right) // 2

    if arr[mid]["id"] == target:
      return arr[mid]
    elif arr[mid]["id"] < target:
      left = mid + 1
    else:
```

```
            right = mid - 1

    return "Crop not found"


# Merge Sort for sorting crops
def merge_sort(arr, key):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid], key)
    right = merge_sort(arr[mid:], key)

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

```
# Sort crops by moisture level
sorted_moisture = merge_sort(crops, "moisture")
print("Sorted by Moisture:", sorted_moisture)

# Sort crops by yield estimate
sorted_yield = merge_sort(crops, "yield")
print("Sorted by Yield:", sorted_yield)

# Search crop by ID
crops_sorted = sorted(crops, key=lambda x: x["id"])
print("Search Result:", binary_search(crops_sorted, 3))
```

Output:



Justification:

Binary Search enables quick retrieval of crop details using *crop ID. Merge Sort helps arrange crops by **soil moisture level or yield estimate*. This improves monitoring and analysis of agricultural data.

Task Description #8: Airport Flight Management System

An airport system stores flight information including flight ID,

airline name, departure time, arrival time, and status. The system

must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

• Use AI to recommend algorithms.

• Justify the algorithm selection.

• Implement searching and sorting logic in Python.

Code:

Generate a Python program for an Airport Flight Management System that stores flight ID, airline name, departure time, arrival time, and status, uses Binary Search to find flights by flight ID, and uses Merge Sort to sort flights by departure time or arrival time with proper comments

Flight records

flights = [

{"id":101, "airline":"IndiGo", "departure":"10:30", "arrival":"12:45", "status":"On Time"},

{"id":102, "airline":"Air India", "departure":"08:15", "arrival":"10:30", "status":"Delayed"},

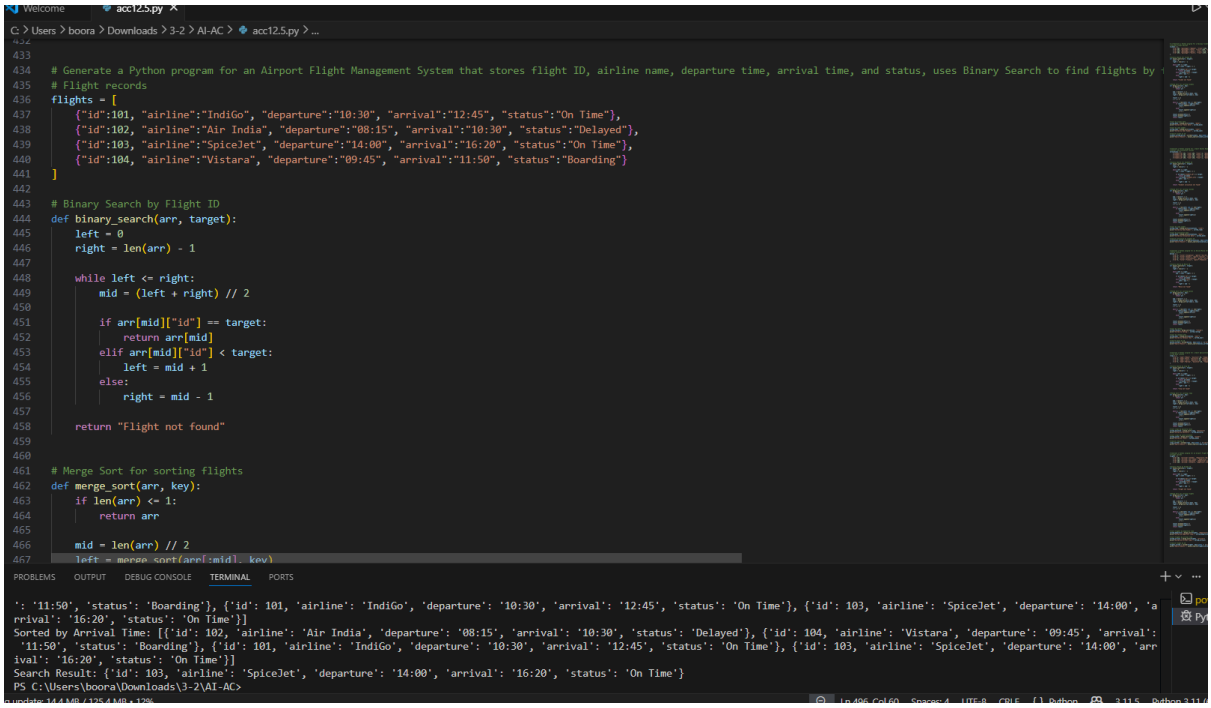
{"id":103, "airline":"SpiceJet", "departure":"14:00", "arrival":"16:20", "status":"On Time"},

```
  {"id":104, "airline":"Vistara", "departure":"09:45", "arrival":"11:50", "status":"Boarding"}
]

# Binary Search by Flight ID
def binary_search(arr, target):
  left = 0
  right = len(arr) - 1

  while left <= right:
    mid = (left + right) // 2

    if arr[mid]["id"] == target:
      return arr[mid]
    elif arr[mid]["id"] < target:
      left = mid + 1
    else:
      right = mid - 1

  return "Flight not found"



# Merge Sort for sorting flights
def merge_sort(arr, key):
  if len(arr) <= 1:
    return arr

  mid = len(arr) // 2
  left = merge_sort(arr[:mid], key)
```

```
    right = merge_sort(arr[mid:], key)

    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i][key] < right[j][key]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result


# Sort flights by departure time
sorted_departure = merge_sort(flights, "departure")
print("Sorted by Departure Time:", sorted_departure)

# Sort flights by arrival time
sorted_arrival = merge_sort(flights, "arrival")
print("Sorted by Arrival Time:", sorted_arrival)

# Search flight by ID
flights_sorted = sorted(flights, key=lambda x: x["id"])
```

print("Search Result:", binary_search(flights_sorted, 103))

Output:



Justification:

Binary Search is used to quickly find flight details using *flight ID. Merge Sort organizes flights based on **departure time or arrival time*. These algorithms help manage flight schedules efficiently.